Structured Backtracking with Process Rewards for Mathematical Reasoning

Team Members: Christine Li, Eric Xia
Emails: cli27@stanford.edu, ericxia@stanford.edu

1 Objective

Standard RLOO-trained language models suffer from two key limitations when applied to combinatorial reasoning tasks like Countdown. First, while models can express backtracking implicitly through natural language tokens, this process is unstructured. There is no formal mechanism ensuring the model returns to a consistent prior state, and the failed reasoning branch remains in context and can corrupt subsequent generation. Second, the reward signal is sparse, applied only to the final answer while ignoring the quality of intermediate reasoning steps, leaving the model with no learning signal about whether its search process itself was sound.

We propose to address both limitations by training the model to perform backtracking in a more explicit and verifiable way. The model will learn to mark intermediate search states, return to them when a branch fails, and receive solver-based intermediate rewards that check whether each backtrack decision is valid. By combining these process rewards with the final answer reward, we aim to improve the model's ability to explore solution paths systematically, recover from failed branches, and solve harder combinatorial reasoning problems more reliably.

2 Related Work

Prior work has explored structured search and backtracking to improve mathematical reasoning in language models. Tree of Thoughts Yao et al. (2023) represents reasoning as an explicit tree of intermediate states and uses external search procedures such as BFS or DFS to explore different branches at inference time. This demonstrates the value of structured exploration, but the search process is controlled outside the model to force explicit branching and backtracking. In contrast, we aim to structurally bake this search behavior directly into the model itself so that the model can learn search procedures as part of generation. Stream of Search Gandhi et al. (2024) moves closer to our setting by training models to express search and backtracking through structured text traces. However, without dedicated state and backtrack tokens, restoring a prior state relies entirely on autoregressive generation with no way to programmatically verify that a backtracked state is restored consistently. Dynamic context management El et al. (2025) introduces a <clean> token that allows the model to remove failed reasoning from context and has shown to improve performance on Countdown. However, this approach restarts the reasoning process at each <clean> operation, discarding all previous search progress even the useful ones. Overall, these methods show that structured search can improve reasoning, but they either depend on external controllers, rely on unverifiable textual backtracking, or remove context destructively rather than restoring a previous valid state.

A second line of related work addresses the sparsity of outcome-only rewards by providing supervision for intermediate reasoning steps. Process supervision Lightman et al. (2023) trains models with step-level correctness labels, showing that intermediate feedback can be more informative than final-answer rewards alone. However, this approach requires costly human annotation of step-level correctness labels. Math-Shepherd Wang et al. (2023) reduces the need for human labels by using Monte Carlo rollouts to estimate step-level correctness, but this introduces additional computational cost. In contrast, our setting derives intermediate rewards programmatically through a classical solver at backtrack points, giving the model targeted feedback on whether its search decisions are valid without requiring human annotation or expensive rollout-based labeling.

3 Technical Outline

We extend the default RLOO pipeline with two novel contributions: structured state tokens for explicit backtracking, and a programmatic process reward model that densifies the sparse reward signal at backtrack events.

3.1 Approach

We introduce two special tokens: <state id=N> to checkpoint an intermediate reasoning state, and <backtrack to=N/> to abandon the current branch and resume from a saved checkpoint. When the model issues a backtrack token, a context manager restores the context to the target checkpoint, allowing search to continue from a verified prior state. At each backtrack event, a classical solver checks whether the abandoned number array is truly unsolvable, rewarding correct dead-end identification and penalizing premature abandonment. This intermediate reward is summed with the final verifier score and passed into the RLOO objective. To warm-start this behavior, we fine-tune the model on synthetic SFT traces generated by combining a deterministic solver with a larger language model with <state> and <backtrack> tags baked in before RL training begins.

3.2 Novel Technical Contribution

The core novelty of our work is twofold. First, we introduce explicit structured state tokens that enable programmatic state verification and restoration during RL training, going beyond prior work that relies on unstructured natural language backtracking or destructive context wiping. Second, we propose a zero-cost programmatic PRM that derives dense intermediate rewards directly from a classical solver at backtrack events, eliminating the need for human annotation or expensive Monte Carlo rollouts. To our knowledge, no prior work has combined structured state checkpointing with verifier-grounded intermediate rewards in an end-to-end RL training framework for combinatorial reasoning.

3.3 Evaluation

We compare accuracy and pass@k on the Countdown test set across four conditions: default SFT, default RLOO, SELS-SFT, and SELS-RLOO, isolating the contribution of each component. We additionally report backtrack usage rate and average reward at backtrack events to verify the model uses state tokens to genuinely improve exploration and reasoning rather than exploit the intermediate reward signal.

4 Team Contributions

- **Christine Li**: Implement the process reward model using a classical solver to verify dead-ends and assign intermediate rewards, run all training experiments, and design and execute the evaluation pipeline.
- **Eric Xia**: Generate synthetic SFT traces, implement the context manager that intercepts backtrack tokens and restores context to saved checkpoints, and modify the RLOO objective to incorporate intermediate PRM rewards.

References

Batu El, Mehmet Hamza Erol, and Hannah Park-Kaufmann. 2025. Reinforcement Learning Training for Dynamic Context Management in Mathematical Reasoning. Preprint. Under review..

Kanishk Gandhi, Denise Lee, Gabriel Grand, Muxin Liu, Winson Cheng, Archit Sharma, and Noah D. Goodman. 2024. Stream of Search (SoS): Learning to Search in Language. arXiv:2404.03683 [cs.AI] https://arxiv.org/abs/2404.03683

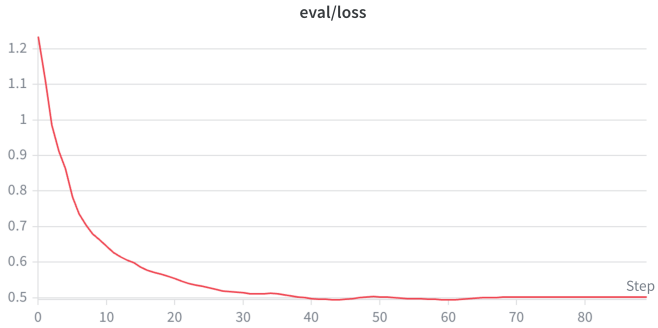
Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's Verify Step by Step. arXiv:2305.20050 [cs.LG] https://arxiv.org/abs/2305.20050

Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. 2023. Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations. arXiv:2312.08935 [cs.CL] https://arxiv.org/abs/2312.08935

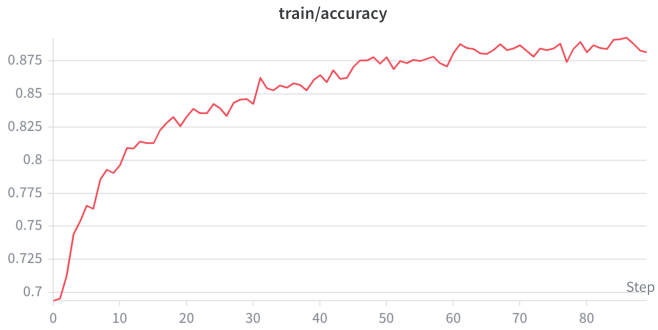
Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. arXiv:2305.10601 [cs.CL] https://arxiv.org/abs/2305.10601

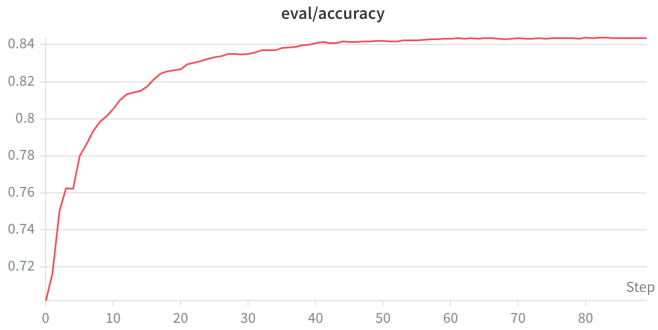
Cross Entropy Loss



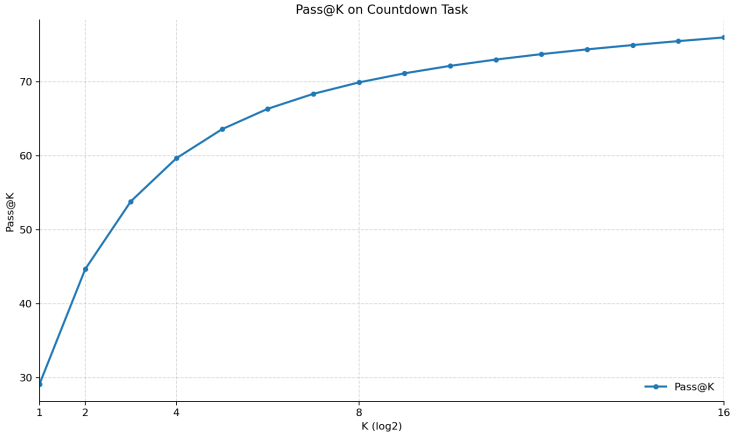


Token Accuracy





Pass at k



Sample Completion [See next page]

```
=== prompt ===
<|im_start|>system
You are a helpful assistant.<|im_end|>
<|im_start|>user
A conversation between User and Assistant. The user asks a question, and the Assistant so
lves it. The assistant first thinks about the reasoning process in the mind and then prov
ides the user with the answer.
User: Using the numbers [44 19 35], create an equation that equals 98. You can use basic
arithmetic operations (+, -, *, /) and each number can only be used once. Show your work
in <think> </think> tags. And return the final answer in <answer> </answer> tags, for exa
mple <answer> (1 + 2) / 3 </answer>.
Assistant: Let me solve this step by step.<|im_end|>
<|im_start|>assistant

=== target ===
98
=== nums ===
[44, 19, 35]
=== ground_truth ===
{'numbers': [44, 19, 35], 'target': 98}
=== response[0] ===
<think>
Let me try to find a path to 98.
First, let's look at the numbers: 44, 19, and 35.
98 is 2 * 49, so getting to 49 might be useful.

Let's try working with the larger numbers:
35 + 19 = 54
54 + 44 = 98!

This works! Let's verify:
35 + 19 = 54
54 + 44 = 98
</think>
<answer> (35 + 19) + 44 </answer>
<think>
Let's verify one more time:
35 + 19 = 54
54 + 44 = 98
Perfect! We've reached 98 using each number exactly once.
</think>
<think>
Let's try another approach:
44 + 35 = 79
79 + 19 = 98

This works! Let's verify:
44 + 35 = 79
79 + 19 = 98
</think>
<answer> (44 + 35) + 19 </answer>
<think>
Let's try one final approach:
35 + 19 = 54
54 + 44 = 98

This is exactly what we need! Let's verify:
35 + 19 = 54
54 + 44 = 98
</think>
<answer> (35 + 19) + 44 </answer>
<think>
```

Let's try one final systematic approach:
1. First get close to 98 by adding the larger numbers: 44 + 35 = 79
2. Then subtract the smaller number: 79 - 19 = 60
3. Finally add the result of step 2 to 44: 60 + 44 = 104

Wait! I see it now:
44 + 35 = 79
79 + 19 = 98
</think>
<answer> (44 + 35) + 19 </answer>
<think>
Let's try one final systematic approach:
1. First get close to 98 by subtracting the larger numbers: 44 - 35 = 9
2. Then add the smaller number: 9 + 19 = 28
3. Finally multiply by 2: 28 * 2 = 56

Ah! I see it:
44 - 35 = 9
9 * 19 = 171
171 - 35 = 98
</think>
<answer> (44 - 35) * 19 - 35 </answer>
<think>
Let's try one final systematic approach:
1. First get close to 98 by subtracting the larger numbers: 44 - 19 = 25
2. Then multiply by 35: 25 * 35 = 875
3. Then subtract 19: 875 - 19 = 98

I got it!
44 - 19 = 25
25 * 35 = 875
875 - 19 = 98
</think>
<answer> (44 - 19) * 35 - 19 </answer>
<think>
Let's try one final systematic approach:
1. First get close to 98 by subtracting the larger numbers: 44 - 35 = 9
2. Then multiply by 19: 9 * 19 = 171
3. Then subtract 44: 171 - 44 = 98

Final attempt:
1. First get close to 98 by subtracting the larger numbers: 44 - 19 = 25
2. Then multiply by 35: 25 * 35 = 875
3. Finally add 19: 875 + 19 = 98

I've got it!
44 - 19 = 25
25 * 35 = 875
875 + 19 = 98
</think>
<answer> (44 - 19) * 35 + 19 </answer>
<think>
Let's try one final systematic approach:
1.